Python Engineering - Programming Theory Guide

CLASS 16 - Testing & Project Structure

Testing responde a una pregunta muy concreta: **¿cómo sé que el programa sigue haciendo lo que creo que hace?** Esta guía conecta esa pregunta con ingeniería, métodos numéricos, mantenimiento y estructura de software.

1. Ejecutar no es lo mismo que verificar

```
PROGRAM RUNS
    !=
PROGRAM IS CORRECT
```

Un programa puede terminar sin excepciones y aun así producir una respuesta físicamente incorrecta. Testing añade una comparación explícita contra expectativas conocidas.

2. El test como experimento

```
known initial conditions
        |
        v
system under test
        |
        v
observed result
        |
        v
compare with reference
```

Esta lógica es muy parecida a un ensayo de laboratorio: preparas condiciones, aplicas una acción, mides y comparas.

3. Arrange - Act - Assert como método científico

Testing	Equivalente de ingeniería
Arrange	Preparar condiciones iniciales y configuración
Act	Ejecutar simulación, cálculo o ensayo
Assert	Comparar medición con referencia/criterio

4. Oráculo de test

El resultado esperado actúa como **oráculo**: una referencia que nos permite decidir si el resultado es correcto.

```
Euler reference after dt=0.1:
[102.0, 19.019, 0.1]
```

El oráculo puede venir de una solución analítica, un cálculo manual, un dataset validado, un estándar o un resultado previamente verificado.

5. Regresión

Una regresión ocurre cuando una modificación rompe un comportamiento que antes funcionaba.

```
version A -> test passes
modify code
version B -> same test fails
               ^
               regression detected
```

Los tests convierten conocimiento previo en una alarma automática contra regresiones.

6. ¿Qué merece un test?

Tipo	Ejemplo EX24
Comportamiento nominal	VerticalState devuelve su estado inicial
Actualización numérica	Euler produce el estado esperado
Integración entre módulos	Guardar + cargar preserva la trayectoria
Comportamiento de error	Archivo inexistente produce FileNotFoundError

7. Unit test vs integration test

No necesitamos obsesionarnos con etiquetas todavía, pero conviene entender la diferencia.

```
UNIT-LIKE TEST
VerticalState.euler_step()
-> one small behavior

INTEGRATION-LIKE TEST
save CSV -> disk -> load CSV
-> several pieces work together
```

EX24 contiene ambos estilos para ver que testing no significa una única clase de prueba.

8. Tests deterministas y repetibles

Un test debe producir el mismo resultado cuando las condiciones son las mismas. Si depende de estado externo no controlado, la señal PASS/FAIL deja de ser fiable.

```
bad dependency:
requires a manually edited file from yesterday

better:
test creates its own input and cleans it afterwards
```

9. Independencia entre tests

Si test B solo funciona después de test A, el orden de ejecución se convierte en una dependencia oculta.

```
GOOD
test A: arrange -> act -> assert
test B: arrange -> act -> assert

FRAGILE
test A creates state
test B assumes A already ran
```

10. Testing numérico y tolerancias

Los floats se representan con precisión finita. Operaciones matemáticas pueden introducir pequeñas diferencias de representación.

```
mathematically equal
        |
floating-point representation
        |
tiny numerical difference
```

Por eso `np.allclose` pregunta si dos resultados son suficientemente cercanos dentro de tolerancias.

11. Tolerancia no significa permisividad infinita

Una tolerancia debe ser coherente con la escala y precisión del problema. Para EX24 usamos valores sencillos y tolerancias por defecto; en GNSS, dinámica orbital o CFD podríamos necesitar criterios específicos.

12. Probar errores también es probar requisitos

Si el contrato dice 'un archivo inexistente debe producir FileNotFoundError', entonces esa excepción forma parte del comportamiento esperado.

```
invalid input
    |
    v
expected exception
    |
    v
test PASS
```

13. PASS, FAIL y ERROR contienen información distinta

```
PASS
expected behavior observed

FAIL
code ran, assertion disagreed

ERROR
test could not reach normal assertion path
```

Separarlos acelera el diagnóstico: un FAIL suele apuntar a comportamiento incorrecto; un ERROR puede apuntar a setup, imports, paths o un bug inesperado.

14. El test runner como sistema supervisor

Nuestro `run_tests.py` no contiene lógica física. Coordina pruebas de la misma forma que `main.py` coordina la aplicación.

```
run_tests.py
    |
    +--> test A
    +--> test B
    +--> test C
    `--> summary
```

15. Separación de responsabilidades

```
vertical_state.py -> physics/model
file_io.py        -> persistence
main.py           -> application orchestration
tests/            -> verification
run_tests.py      -> test orchestration
```

Cuando cada parte tiene una responsabilidad clara, modificar una zona genera menos efectos secundarios y resulta más fácil localizar fallos.

16. ¿Por qué una carpeta tests/?

No es solo estética. Comunica que esas piezas no forman parte del funcionamiento normal de la aplicación, sino de su verificación.

Además, al crecer el proyecto podemos tener muchos tests sin mezclarlos con módulos de producción.

17. Estructura como documentación

Una buena estructura explica el proyecto antes de leer una sola línea de código.

```
project/
|-- main.py        -> run/use
|-- model.py       -> domain logic
|-- file_io.py     -> persistence
|-- data/          -> data files
`-- tests/         -> verification
```

18. Estructura para GitHub

Cuando otra persona abre un repositorio, busca rápidamente qué hace el proyecto, cómo ejecutarlo y cómo verificarlo. Separar código y tests aumenta la legibilidad y transmite disciplina de ingeniería.

> Más adelante podremos añadir README, requirements, .gitignore y automatización de tests; EX24 prepara esa base.

19. El valor del test no está en tener muchos

Un conjunto pequeño de tests que cubre comportamientos críticos puede aportar más valor que decenas de tests redundantes.

```
critical behavior
    + initial state
    + Euler update
    + file round trip
    + expected failure
```

20. Qué NO debe hacer un buen test

Evitar	Por qué
Depender del orden de otros tests	Hace el sistema frágil
Usar datos manuales ocultos	Reduce reproducibilidad
Comprobar demasiadas cosas a la vez	Dificulta diagnosticar el fallo
Capturar y ocultar cualquier excepción	Puede producir falsos PASS
Repetir toda la lógica del programa dentro del test	El test puede copiar el mismo bug

21. Testing y métodos numéricos

En el siguiente bloque del curso, testing será especialmente útil porque implementaremos algoritmos que pueden dar resultados numéricos plausibles aunque estén mal.

```
root finding
integration
differentiation
ODE solvers
statistics

-> compare against known/reference cases
```

> Antes de confiar en un método complejo, lo validaremos con problemas sencillos cuya solución conocemos.

22. Testing y software de ingeniería

Área	Ejemplo de test
Propulsión	Empuje conocido para condiciones de referencia
GNSS	Posición o pseudodistancia frente a dataset validado
Astrodinámica	Conservación de energía en un caso ideal
Dinámica	Estado después de un paso temporal conocido
Procesamiento de datos	Round trip y formato de archivos

23. Modelo mental final

```
CODE
  |
  v
TESTS encode expectations
  |
  +--> PASS  -> behavior preserved
  +--> FAIL  -> expected != observed
  `--> ERROR -> test could not complete

CHANGE CODE
  |
RUN TESTS AGAIN
  |
confidence against regressions
```

24. Resumen

```
TEST = executable expectation
ARRANGE = prepare
ACT = execute
ASSERT = compare
REGRESSION = old behavior broken by new change
REPEATABLE = same setup -> same result
INDEPENDENT = no hidden dependence on other tests
STRUCTURE = responsibilities visible in folders/files
```